

SYSTEMANALYSE UND ENTWURF

CineTicket

Ein Kino-Ticketsystem. Von der Anforderungsanalyse über die UML-Modelle bis zum klickbaren Prototyp.

 linusk100.github.io/Kino-Project-Vorbereitung

CINETICKET

So ist das Projekt gebaut

Alle Inhalte sind strukturierte Daten

`{ } states.json`

```
{
  "from": "RESERVIERT",
  "to": "BELEGT",
  "event": "belegen(buchung)"
}
```



LIVE ALS SVG

RESERVIERT

belegen(buchung)

BELEGT

- Personas, Stories, UML und Zustände liegen als JSON-Dateien vor

- Die Diagramme werden daraus live als SVG gerendert

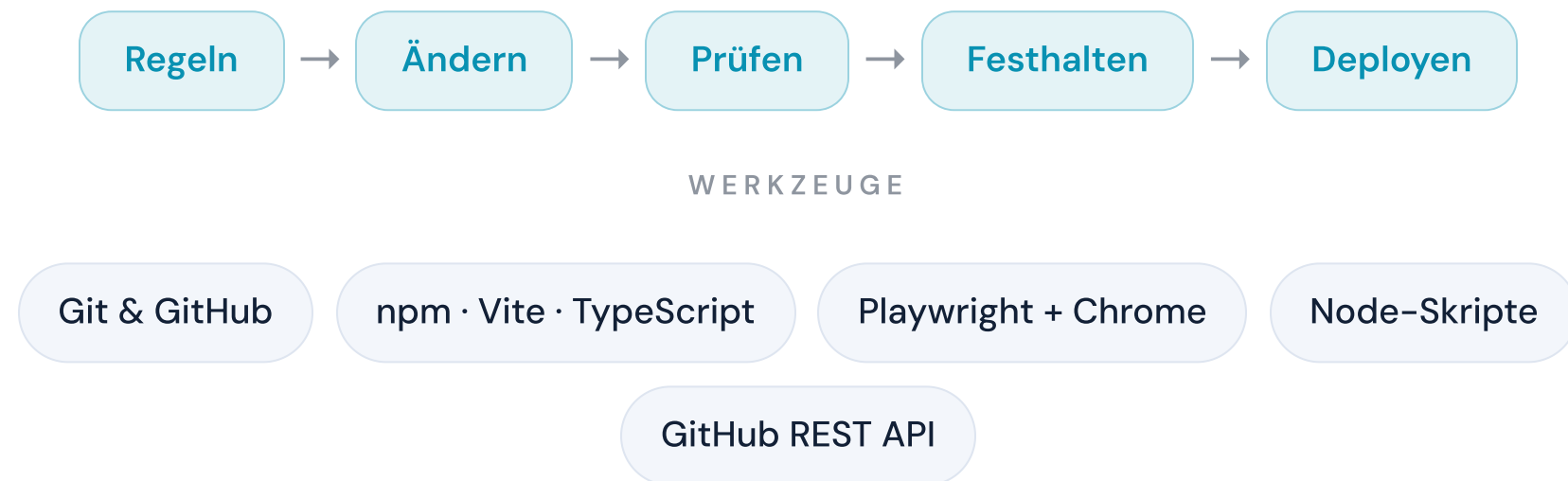
- Inhalt und Darstellung können nicht auseinanderlaufen

Zwei Detailgrade auf der Website: Einfach ist immer eine echte Teilmenge von Erweitert.

CINETICKET

So ist es entstanden

Fester Ablauf, Standard-Werkzeuge



- Jede Änderung: Regeln lesen, ändern, prüfen, festhalten, deployen

- Playwright testet die echte Seite im Browser

- Node-Skripte prüfen die Datenkonsistenz

Nichts geht ungeprüft live: Lint, Build und Browser-Test sichern jeden Stand.

CINETICKET

Agenda

Sechs Stationen bis zur Diskussion

1 **Projekt & Vorgehen**
Datenbasis und Arbeitsweise

2 **Anforderungen**
Personas, User Stories, Story Map

3 **Modellierung**
Klassen, Sequenzen, Zustände

4 **Prototyp**
Live-Demo im Browser

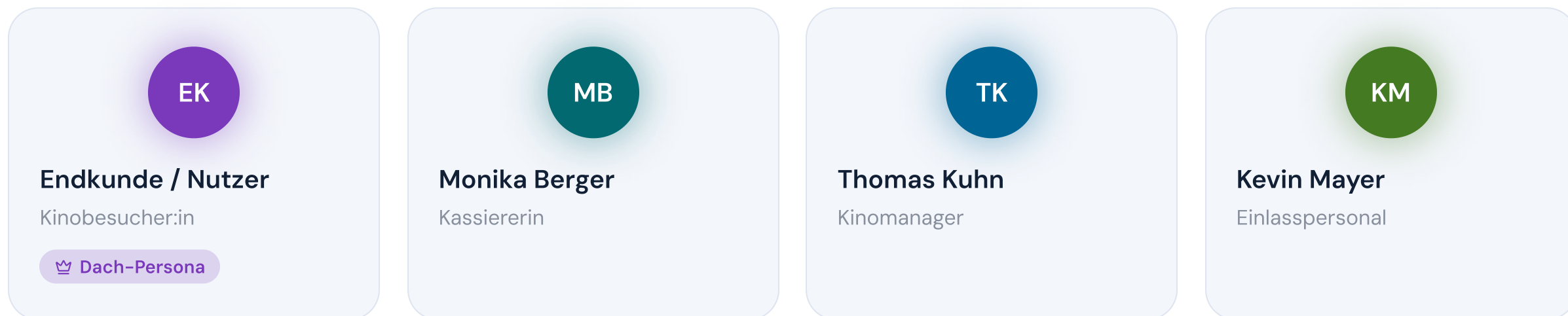
5 **Innovation**
Sechs Ideen nach dem MVP

6 **Fazit**
Der Weg zum fertigen System

PERSONAS

Kundschaft und Betrieb

Eine Rolle draußen, drei drinnen



- Der Endkunde bucht Tickets, mobil oder im Web

- Im Betrieb: Monika an der Kasse, Thomas als Manager, Kevin am Einlass

- Verschiedene Ziele ergeben verschiedene Anforderungen

Jede User Story gehört zu genau einer Persona.

PERSONAS

So tickt die Kundschaft

Schnell buchen, den Lieblingsplatz sichern



Endkunde / Nutzer

Kinobesucher:in · 35 Jahre

„In 60 Sekunden gebucht, der Lieblingsplatz reserviert.“

🎯 ZIELE

- Schnell und unkompliziert Tickets kaufen (mobil unter 60 Sekunden)
- Den besten freien Platz per KI-Vorschlag mit einem Tipp übernehmen
- Rabatte (Senior, Student, Kind, Gruppe) ohne Hürden anwenden
- Barrierefreie Plätze und Begleitperson-Optionen klar erkennen
- Bonuspunkte sammeln, einsehen und einlösen

😞 FRUSTRATIONEN

- Lange Ladezeiten und komplizierte Mobile-Buchung
- Sitzplan ohne Komfort- oder Barrierefreiheits-Details
- Gruppen-Tickets nur einzeln buchbar
- Keine modernen Zahlungsmittel (Apple Pay / Google Pay)

• Frustrationen: lange Ladezeiten, Gruppen-Tickets nur einzeln buchbar

• Daraus entstehen der Buchungs-Wizard und der Sitz-Hold

PERSONAS

So tickt der Betrieb

An der Kasse zählt Tempo



Monika Berger

Kassiererin · 38 Jahre

„Schnell, korrekt, freundlich – in dieser Reihenfolge.“

 ZIELE

- Tickets schnell und fehlerfrei verkaufen
- Warteschlangen minimieren
- Einfache Stornierungen durchführen

 FRUSTRATIONEN

- Langsame Ladezeiten im System
- Komplizierte Sitzplatzbuchung unter Zeitdruck
- Keine Übersicht bei ausverkauften Vorstellungen

- Monika will schnell und fehlerfrei verkaufen

- Frustration: langsame Systeme im Stoßbetrieb

- Daraus entsteht der Schnellverkauf mit Tastatur-Shortcuts

Im Vollausbau werden aus den Rollen konkrete Profile, von der Stammkundin bis zur Familienmutter.

USER STORIES

Ein Satz, drei Antworten

Wer braucht was, und wozu?

Als **Persona** möchte ich **Ziel** um **Nutzen** .

Persona – wer braucht es?

Ziel – was soll möglich sein?

Nutzen – warum lohnt es sich?

- Als Persona möchte ich Ziel, damit Nutzen

- Akzeptanzkriterien machen testbar, wann eine Story fertig ist

- Jede Story trägt eine MoSCoW-Priorität und ein Release

Jede Story kennt ihre Persona, ihre Aktivität und ihr Release.

USER STORIES

So verschieden sind die Stories

Vier Beispiele: verschiedene Personas, Prioritäten und Releases

U47 **DER KERN** **Hohe Priorität** **Release 1 – MVP** **LH** Lara

Als Stammkundin möchte ich, dass mein gewählter Platz beim Checkout verbindlich für eine begrenzte Zeit reserviert wird, damit ihn niemand parallel buchen kann (keine Doppelbuchung).

U01 **Hohe Priorität** **Release 1 – MVP** **MB** Monika

Als Monika möchte ich den Schnellverkaufsmodus mit einem Klick öffnen, um Tickets ohne Navigation durch Menüs zu verkaufen.

U15 **Hohe Priorität** **Release 1 – MVP** **KM** Kevin

Als Kevin möchte ich Tickets per QR-Code-Scan validieren, damit der Einlass schnell und sicher abläuft.

U22 **Mittlere Priorität** **Release 2 – Erweiterung** **JS** Jonas

Als Student möchte ich das aktuelle Programm nach Genre, Datum und Uhrzeit filtern, um passende Vorstellungen zu finden.

• Von der Kasse über den Einlass bis zur Komfort-Suche

• U47, der Sitz-Hold, ist der fachliche Kern: er verhindert Doppelbuchungen

• Akzeptanzkriterien öffnen sich auf der Website je Karte

U47 taucht wieder auf: im Sequenzdiagramm und im Zustandsautomaten.

Waagrecht die Nutzerreise, senkrecht die Releases

NUTZERREISE →	TICKET KAUFEN	PROGRAMM & SUCHE	SITZPLAN & KATEGORIEN	EINLASS & VALIDIERUNG	GASTRO & SERVICE	FACILITY & WARTUNG	VORSTELLUNG VERWALTEN	KONTO & LOYALITÄT	SYSTEM & ADMIN	MARKE & MULTI-SITE
Release 1 · MVP	8	–	2	2	–	–	2	1	1	–
Release 2	1	4	5	1	3	2	1	3	5	2
Release 3	–	2	–	1	1	–	2	1	–	1

Zahl = User Stories der Aktivität im Release · gestrichelte Aktivitäten kommen im Vollausbau dazu

- Oben die Aktivitäten, vom Ticketkauf bis zur Verwaltung
- Release 1 ist die schmale, lauffähige Scheibe: kaufen, Sitz
- Suche, Gastro, Facility und Marke folgen in Release 2 und 3

- Release 1 ist die schmale, lauffähige Scheibe: kaufen, Sitz wählen, einlösen, verwalten

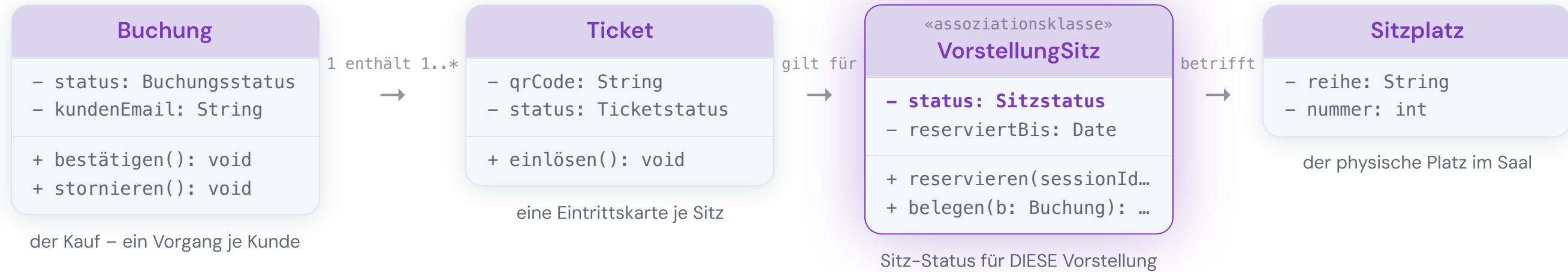
- Suche, Gastro, Facility und Marke folgen in Release 2 und 3

Auf der Website ist die Karte interaktiv: jede Karte öffnet ihre Story.

KLASSENDIAGRAMM

Das Herzstück: eine Buchung

Vier Klassen, vier klare Aufgaben



- Die Buchung ist der Kauf und enthält je Platz ein Ticket

- Jedes Ticket gilt für einen VorstellungSitz

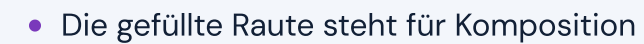
- Der physische Sitzplatz im Saal bleibt unberührt

Statuswechsel sitzen am Objekt selbst. Der BuchungService steuert nur und hält keine Daten.

Eine Assoziationsklasse trennt Sitz und Status



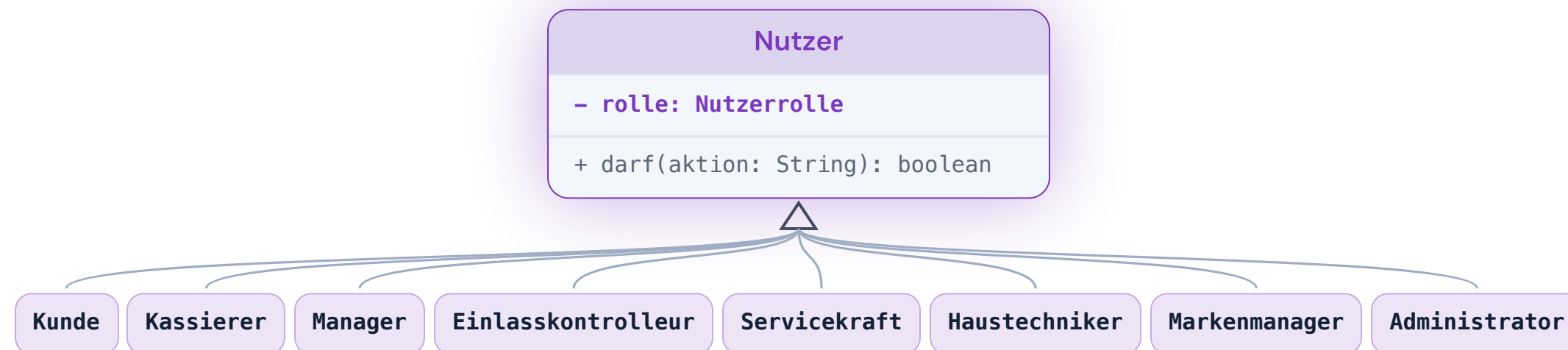
Der Teil existiert nicht ohne das Ganze



KLASSENDIAGRAMM

Rollen erben von Nutzer

Alle Rollen erben von Nutzer



- Vom Kunden bis zum Administrator

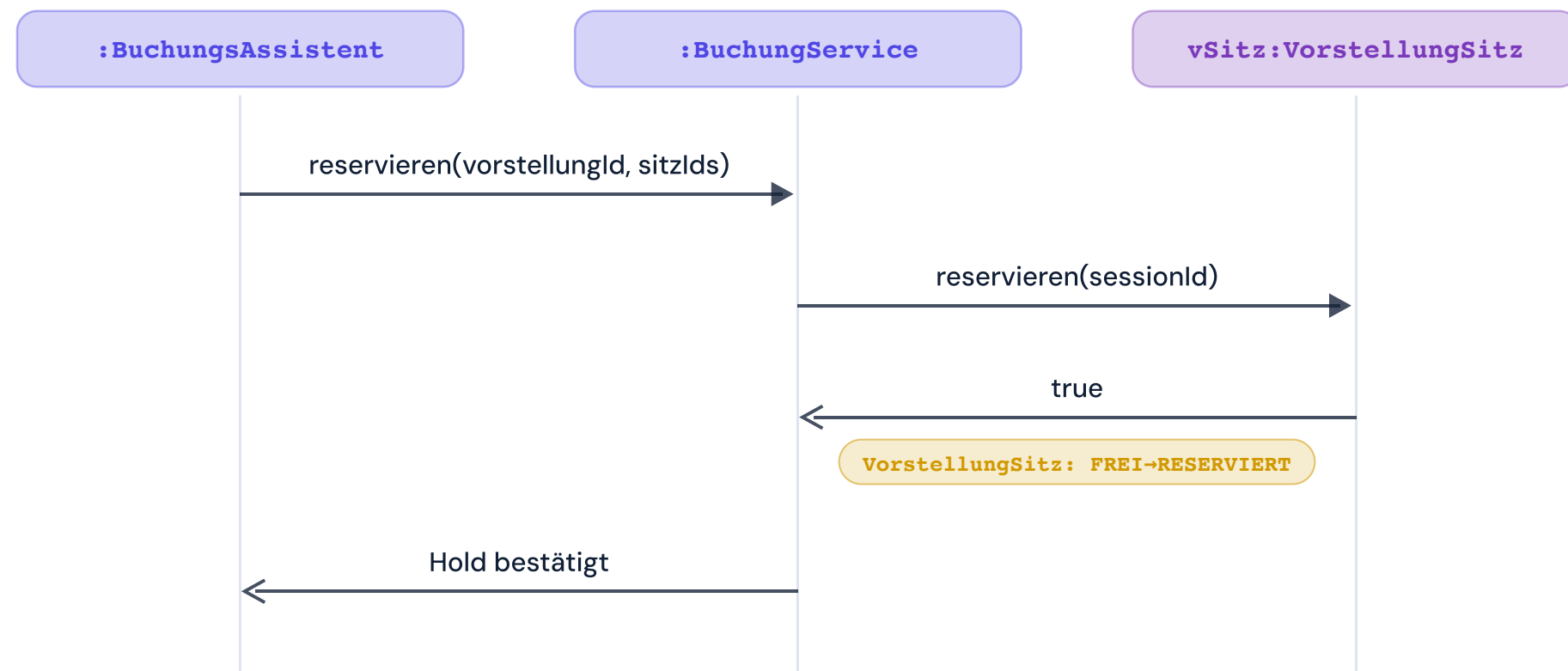
- Rechte entscheidet das Enum Nutzerrolle

- Geprüft an einer Stelle: `darf(aktion)`

SEQUENZDIAGRAMME

Der Hold: 10 Minuten verbindlich

Beim Checkout wird der Sitz serverseitig reserviert



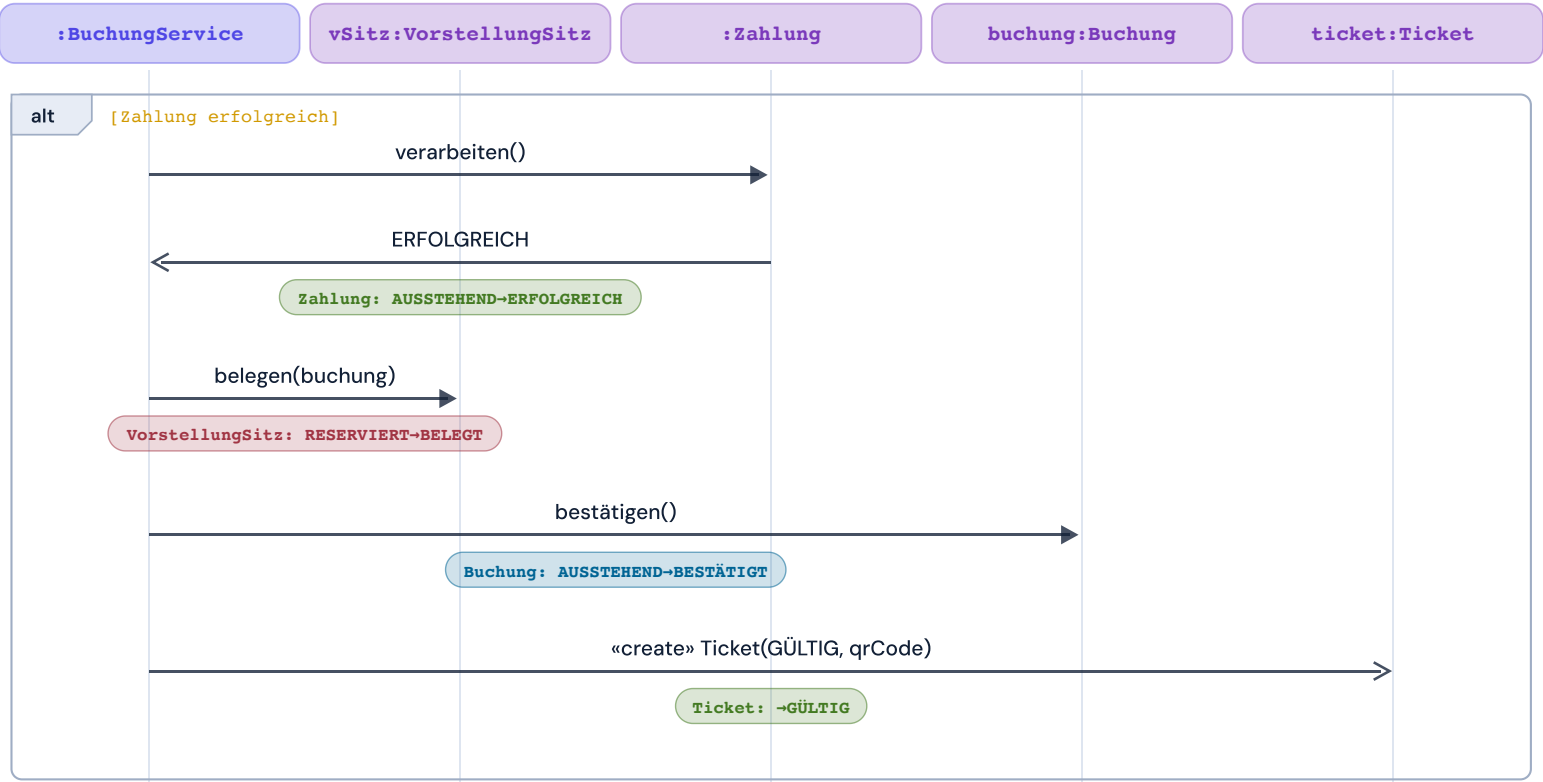
- Der BuchungService ruft reservieren() am VorstellungSitz auf
- Der Status wechselt auf RESERVIERT, parallele Käufer sind blockiert
- Das Badge zeigt den Statuswechsel aus den Daten

Kommt reservieren() zu spät, antwortet der Sitz mit false. Der Kunde landet in der Sitzwahl, nie in einer Doppelbuchung.

SEQUENZDIAGRAMME

Die Zahlung entscheidet

Erst die Zahlung macht aus dem Hold einen Kauf



• Zahlung erfolgreich, Sitz belegt, Buchung bestätigt, Ticket erzeugt

• Vier Statuswechsel in fester Reihenfolge

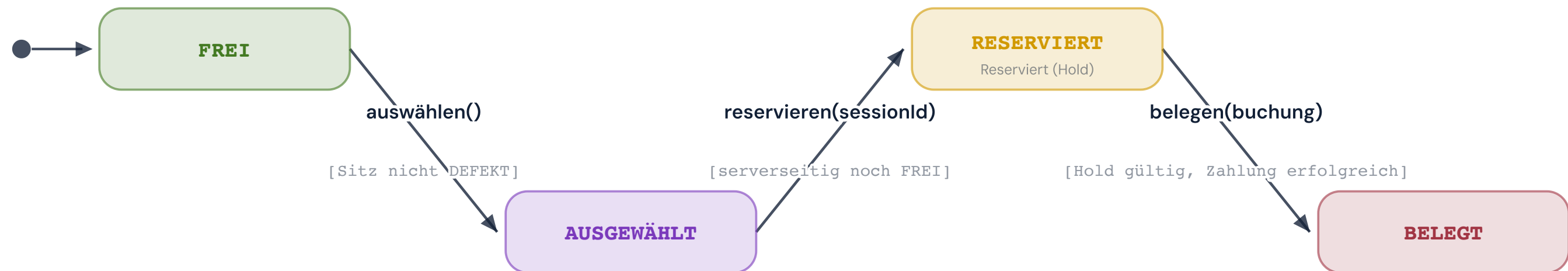
• Das alt-Fragment zeigt den bedingten Zweig

Der Storno läuft genauso als Kette: Buchung storniert, Ticket ungültig, Zahlung erstattet, Sitz wieder frei.

ZUSTANDSDIAGRAMME

Der wichtigste Automat: der Sitz

Ein Automat beschreibt genau ein Objekt über die Zeit



- `auswählen()` ist nur lokal im Browser

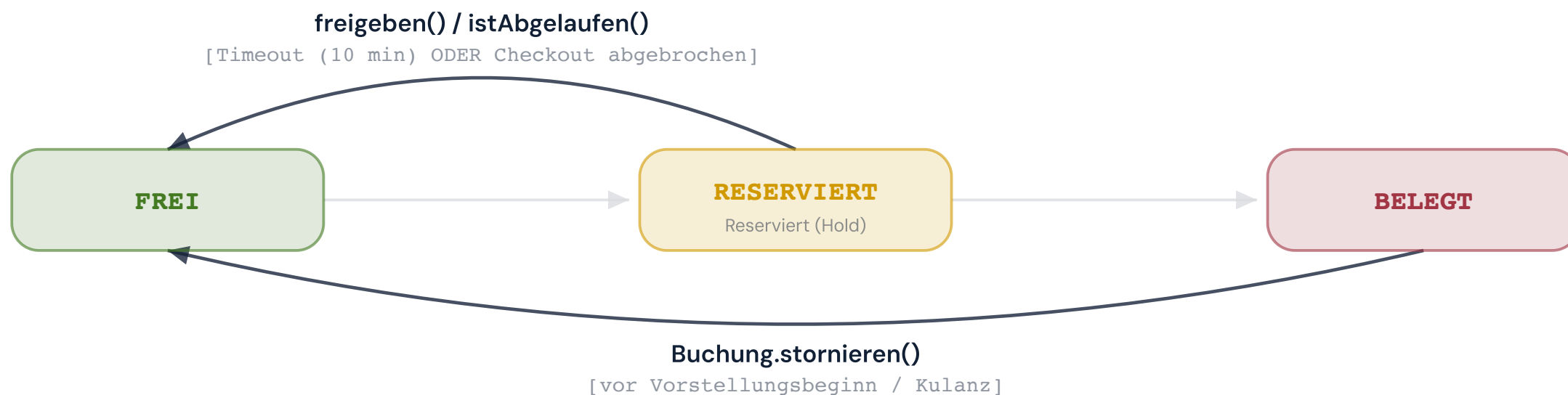
- `reservieren()` setzt den serverseitigen Hold

- `belegen()` erst nach erfolgreicher Zahlung

ZUSTANDSDIAGRAMME

Jeder Hold endet von selbst

Kein Sitz bleibt für immer blockiert



- Hold abgelaufen oder Kauf abgebrochen: automatisch zurück auf FREI

- Ein Storno gibt auch belegte Plätze frei

- DEFEKT sperrt systemweit und erzeugt ein Wartungsticket

Jeder Automat ist wertgleich mit seinem Status-Enum im Klassendiagramm.

ZUSTANDSDIAGRAMME

Die weiteren Automaten in Kürze

Dasselbe Muster für Ticket, Buchung und Zahlung

Ticket-Lebenszyklus

4 Zustände

Ein Anfang, drei Enden: eingelöst, storniert oder abgelaufen.

Buchungs-Lebenszyklus

4 Zustände

Hält alles zusammen. Bestätigt erst mit erfolgreicher Zahlung.

Zahlungs-Lebenszyklus

4 Zustände

Entscheidet den Rest. Erst ein Storno macht daraus ERSTATTET.

PROTOTYP

Live-Demo: der Prototyp

Der Kern des Modells läuft im Browser



4 von 8 Rollen und 41 von 82 UML-Klassen sind gebaut.

- Buchen im Sitzplan, verkaufen an der Kasse

- Steuern im Manager-Cockpit, scannen am Einlass

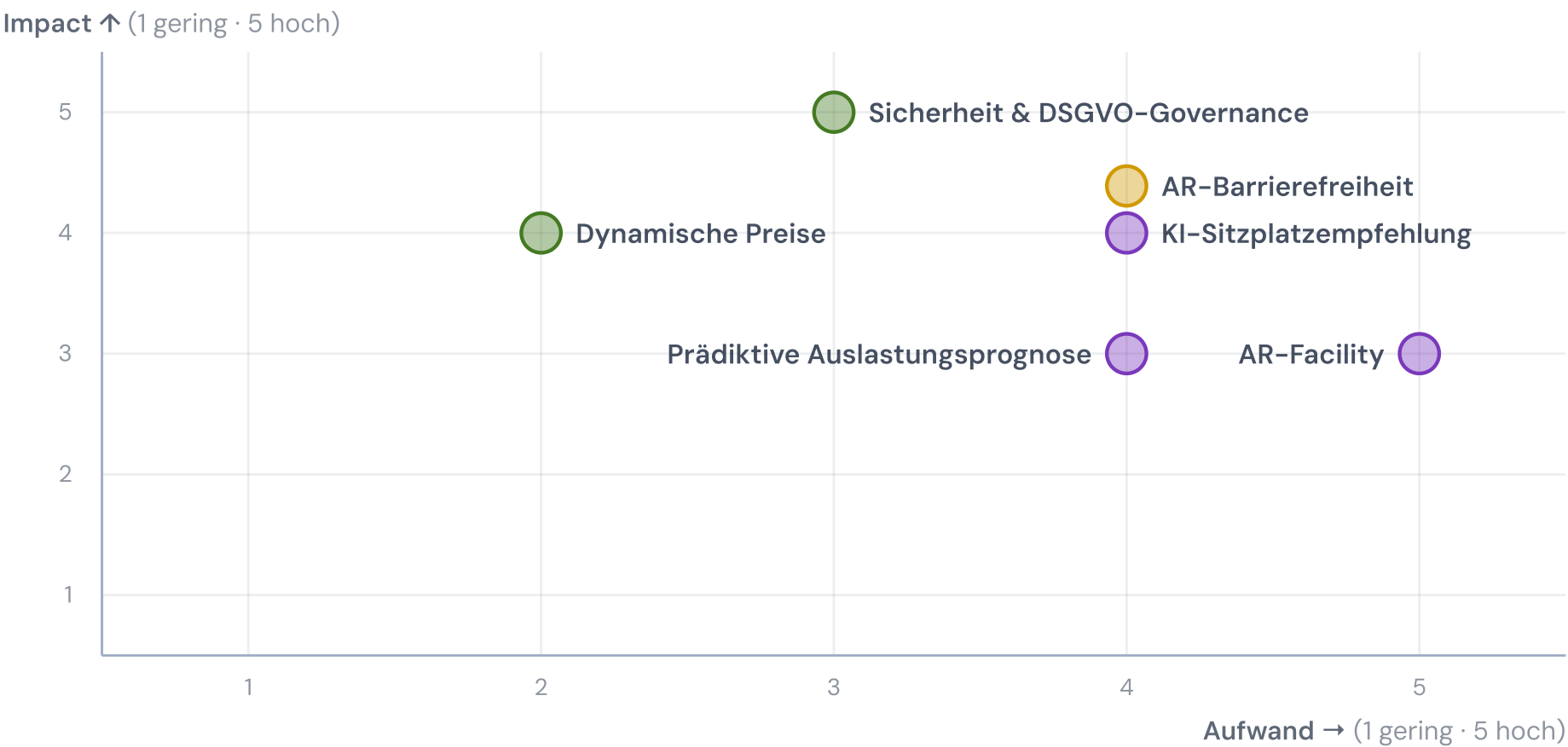
- Vier weitere Rollen sind als Roadmap modelliert

Modell $\supset$ Prototyp: Was fehlt, ist im Klassendiagramm bereits verortet.

INNOVATION

Sechs Ideen, nach Wirkung und Aufwand

Impact gegen Aufwand, Machbarkeit als Farbe



im Rahmen machbar

teilweise machbar

Konzept / Vision

INNOVATION

Nah am MVP

Drei Ideen docken direkt am System an

Dynamische Preise

im Rahmen machbar

Ticketpreise passen sich regelbasiert an die Auslastung an, um Nachfragespitzen besser zu nutzen.

Sicherheit & DSGVO-Governance

im Rahmen machbar

Least-Privilege-Rollen, lückenloses Audit-Log, erzwungene 2FA sowie DSGVO-Auskunft und -Löschung.

AR-Barrierefreiheit

teilweise machbar

Zu einer Vorstellung lässt sich eine Untertitel- oder Audiodeskriptions-Brille dazubuchen.

INNOVATION

Drei Ideen für später

Bewusst als Konzept markiert, recherchiert und eingeordnet

KI-Sitzplatzempfehlung

Konzept / Vision

Die KI schlägt den besten freien Platz passend zu Präferenzen vor – kein langes Suchen im Sitzplan.

AR-Facility: Defekt melden & Sitz sperren

Konzept / Vision

Haustechnik meldet einen defekten Sitz per AR-Brille mit Foto und Sprachnotiz; sofort entsteht ein Wartungsticket und der Sitz wird gesperrt.

Prädiktive Auslastungsprognose

Konzept / Vision

Die erwartete Auslastung kommender Vorstellungen wird prognostiziert, um Programm und Personal vorausschauend zu planen.

ABSCHLUSS

Fazit: der Weg zum fertigen System

Mit diesen Artefakten und agentischem Coding wird aus dem Entwurf das echte System

Daten statt Dokumente

12 Personas, 51 Stories, 82 Klassen als JSON.
Ein Coding-Agent liest sie direkt: als
Datenbankschema, Seed-Daten und Testfälle.

Ein roter Faden

U47 führt von der Story über die Sequenz bis
zum Automaten. Jede Änderung lässt sich bis
zur Anforderung zurückverfolgen.

Spezifikation statt Prompt

Modell, Automaten und Akzeptanzkriterien
sind präzise genug, dass Agenten daraus
Backend und Features bauen. Aus den Enums
und Sequenzen werden die Tests.

Der Ablauf sichert die Qualität

Feste Regeln, echte Browser-Tests,
Gegenprüfung vor jedem Stand. So ist diese
Website entstanden, so entsteht auch das
fertige System.

Anforderungen →

Modell →

Prototyp →

agentisch umgesetzt: das volle System

SYSTEMANALYSE UND ENTWURF

Vielen Dank

Fragen und Diskussion. Alle Details, Diagramme und der Prototyp stehen auf der Projekt-Website.

🌐 linusk100.github.io/Kino-Project-Vorbereitung